

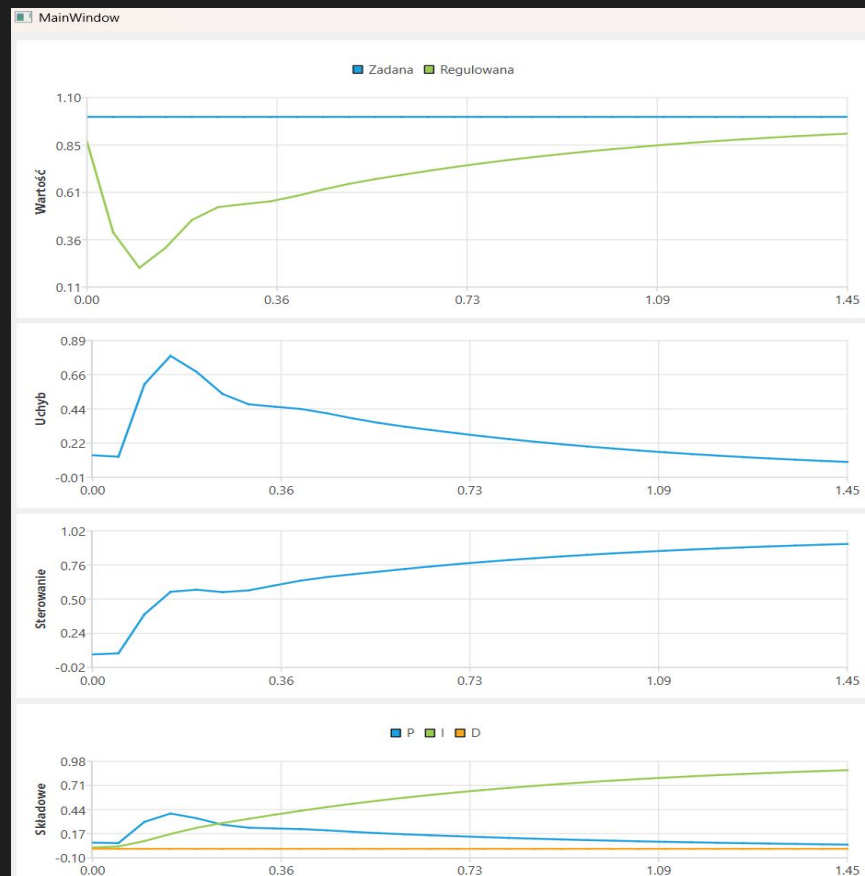
Projekt SK

Skład sekcji:

Kacper Bieniek, Daniel Byra, Natalia Kuśka

Weryfikacja programu

- Brak zerowania buforów sygnałowych w modelu ARX w momencie resetu symulacji

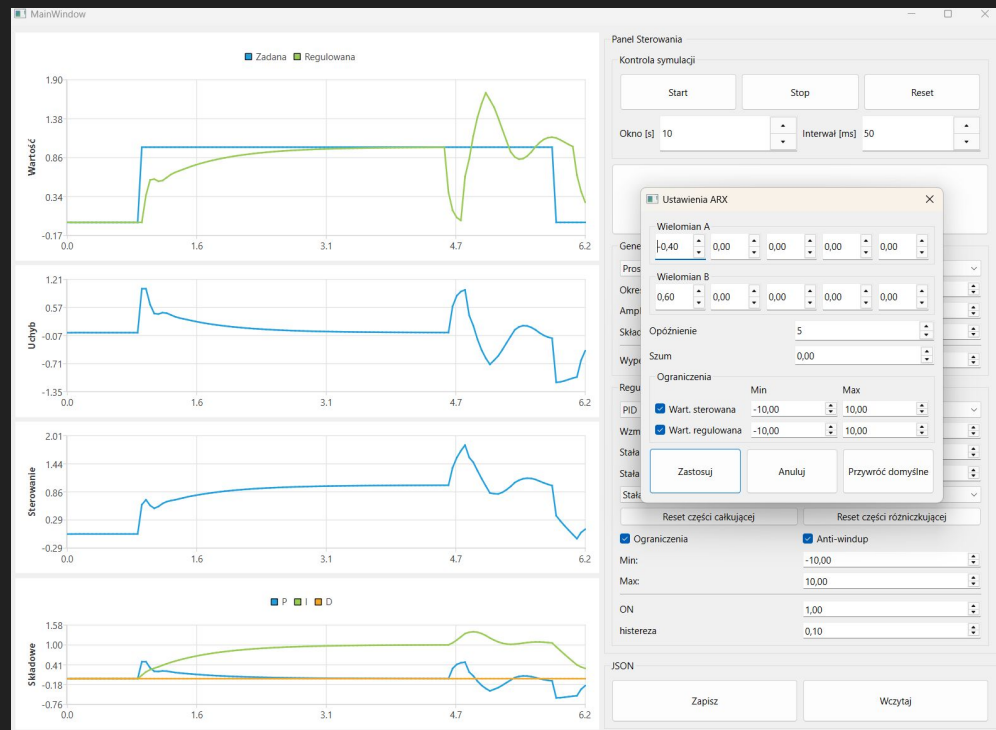


Walidacja programu

- Brak opisu osi X
- Brak szczególnego
wyeksponowania w programie
wykresów związanych z wartością
zadaną i regulowaną



- Opóźnienie transportowe przyjmowane jest dopiero za drugim razem (należy 2 kliknąć Zastosuj, aby symulacja zareagowała)



Ocena programu

Mocne strony:

- Czytelny interfejs
- Czytelny kod

Słabe strony:

- Brak czyszczenia buforu ARX
- Słaby wydajnościowo mechanizm usuwania punktów z wykresów

Omówienie protokołu sieciowego

Enumerate'y i Struktury

```
enum typ_wiadomosci {
    ramka_symulacji = 1,    //(sygnał sterujący, wartość regulowana, wartość zadana)
    komenda_sterujaca = 2, //(start, stop, reset, zmiana interwału)
    konfiguracja_uar = 3    //(parametry)
};

enum typ_serializacji {
    binarna,
    tekstowa
};

struct dane_symulacji {
    double sygnal_sterujacy; // u
    double wartosc_regulowana; // y
    double wartosc_zadana; // w
};

struct dane_sterujace {
    QString polecenie; // START, STOP, RESET
    int interwal;
};

struct dane_konfiguracyjne {
    QString ustawienia_pid;
    QString ustawienia_arx;
    QString ustawienia_generatora;
};
```

Uruchomienie licznika przy wywołaniu konstruktora

```
communication_protocol::communication_protocol(QObject *parent)
    : QObject(parent), aktualny_typ_serializacji(binarna),
      liczba_pakietow(0), aktualne_pps(0.0), czas_ostatniego_pakietu_ms(0) {

    stoper_wydajnosci.start(); // QElapsedTimer
}
```

Sekcja public

```
typ_serializacji podaj_typ_serializacji() const;

//pakiet danych dla ramki symulacji
QByteArray stworz_wiadomosc_symulacji(uint32_t numer_probki, const dane_symulacji& dane);

//pakiet z poleceniem sterującym
QByteArray stworz_wiadomosc_sterujaca(uint32_t numer_probki, const dane_sterujace& dane);

//pakiet z ustawieniami całego przebiegu
QByteArray stworz_wiadomosc_konfiguracyjna(uint32_t numer_probki, const dane_konfiguracyjne& dane);

// główny parser strumienia TCP
void przetworz_strumien(QByteArray& surowe_dane);

//! po wysłaniu dla synchronizacji
void zarejestruj_wyslanie_pakietu();

//! po odebraniu dla synchronizacji
void zarejestruj_odebranie_pakietu();

double podaj_pakiety_na_sekunde() const;
```

Sekcja private

```
private:
    typ_serializacji aktualny_typ_serializacji;

    // pod pomiar wydajności
    QElapsedTimer stoper_wydajnosci; //precyzyjniejsze niż Timer czytelniejsze jednostki - nie zmieniać
    int liczba_pakietow;
    double aktualne_pps; // pps - pakiety na sekunde btw

    // taki long long ale QElapsedTimer go zwraca
    qint64 czas_ostatniego_pakietu_ms;

    // dodawanie nagłówka
    QByteArray stworz_naglowek(typ_wiadomosci typ, uint32_t numer_probki, uint32_t rozmiar_danych);

    dane_symulacji parsuj_dane_symulacji(const QByteArray& dane);
    dane_sterujace parsuj_dane_sterujace(const QByteArray& dane);
    dane_konfiguracyjne parsuj_dane_konfiguracyjne(const QByteArray& dane);
};
```

Tworzenie nagłówka pakietu

```
QByteArray communication_protocol::stworz_naglowek(typ_wiadomosci typ,
                                                    uint32_t numer_probki,
                                                    uint32_t rozmiar_danych) {
    QByteArray naglowek;
    if (aktualny_typ_serializacji == binarna) {

        QDataStream strumien(&naglowek, QIODevice::WriteOnly);
        strumien << (quint32)typ << rozmiar_danych << numer_probki;
    } else {

        QTextStream strumien(&naglowek, QIODevice::WriteOnly);
        strumien << (uint)typ << " " << rozmiar_danych << " " << numer_probki
        | << "\n";
        // łatwiej będzie znaleźć koniec nagłówka -"\n"
    }
    return naglowek;
}
```

Tworzenie wiadomości

```

QByteArray communication_protocol::stworz_wiadomosc_symulacji(uint32_t numer_probki,
    const dane_symulacji &dane) {
    QByteArray czyste_dane;
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(&czyste_dane, QIODevice::WriteOnly);
        strumien << dane.signal_sterujacy << dane.wartosc_regulowana
            << dane.wartosc_zadana;
    } else {
        QTextStream strumien(&czyste_dane, QIODevice::WriteOnly);
        strumien << dane.signal_sterujacy << ";" << dane.wartosc_regulowana << ";"
            << dane.wartosc_zadana;
        strumien.flush(); // trzeba zamknąć
    }

    // kazda wiadomość musi mieć nagłówek
    QByteArray naglowek =
        stworz_naglowek(ramka_symulacji, numer_probki, czyste_dane.size());
    return naglowek + czyste_dane;
}

```

Parser Wiadomości

```
communication_protocol::dane_symulacji communication_protocol::parsuj_dane_symulacji(const QByteArray &dane) {
    dane_symulacji wynik_rozkodowania = {0.0, 0.0, 0.0};
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(dane);
        strumien >> wynik_rozkodowania.signal_sterujacy >>
            wynik_rozkodowania.wartosc_regulowana >>
            wynik_rozkodowania.wartosc_zadana;
    } else {
        QString zwykly_tekst = QString::fromUtf8(dane);
        QStringList wartosci = zwykly_tekst.split(';');
        if (wartosci.size() >= 3) {
            wynik_rozkodowania.signal_sterujacy = wartosci[0].toDouble();
            wynik_rozkodowania.wartosc_regulowana = wartosci[1].toDouble();
            wynik_rozkodowania.wartosc_zadana = wartosci[2].toDouble();
        }
    }
    return wynik_rozkodowania;
}
```

Analogicznie dla konfiguracji oraz sterowania

```
// to samo dla danych sterujacych - inna struktura
QByteArray communication_protocol::stworz_wiadomosc_sterujaca(uint32_t numer_probki,
                                                             const dane_sterujace &dane) {
    QByteArray czyste_dane;
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(&czyste_dane, QIODevice::WriteOnly);
        strumien << dane.polecenie << dane.interwal;
    } else {
        QTextStream strumien(&czyste_dane, QIODevice::WriteOnly);
        strumien << dane.polecenie << ";" << dane.interwal;
        strumien.flush();
    }

    QByteArray naglowek =
        stworz_naglowek(komenda_sterujaca, numer_probki, czyste_dane.size());
    return naglowek + czyste_dane;
}

// to samo dla danych sterujacych - inna struktura
communication_protocol::dane_sterujace
communication_protocol::parsuj_dane_sterujace(const QByteArray &dane) {
    dane_sterujace wynik_rozkodowania;
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(dane);
        strumien >> wynik_rozkodowania.polecenie >> wynik_rozkodowania.interwal;
    } else {
        QString zwykly_tekst = QString::fromUtf8(dane);
        QStringList wartosci = zwykly_tekst.split(';');
        if (wartosci.size() >= 2) {
            wynik_rozkodowania.polecenie = wartosci[0];
            wynik_rozkodowania.interwal = wartosci[1].toInt();
        }
    }
    return wynik_rozkodowania;
}
```

```
QByteArray communication_protocol::stworz_wiadomosc_konfiguracyjna(
    uint32_t numer_probki, const dane_konfiguracyjne &dane) {
    QByteArray czyste_dane;
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(&czyste_dane, QIODevice::WriteOnly);
        strumien << dane.ustawienia_pid << dane.ustawienia_arx
            << dane.ustawienia_generatora;
    } else {
        QTextStream strumien(&czyste_dane, QIODevice::WriteOnly);
        // inny separator by sie nie mieszało - sajgon
        strumien << dane.ustawienia_pid << "|" << dane.ustawienia_arx << "|"
            << dane.ustawienia_generatora;
        strumien.flush();
    }

    QByteArray naglowek =
        stworz_naglowek(konfiguracja_uar, numer_probki, czyste_dane.size());
    return naglowek + czyste_dane;
}

communication_protocol::dane_konfiguracyjne
communication_protocol::parsuj_dane_konfiguracyjne(const QByteArray &dane) {
    dane_konfiguracyjne wynik_rozkodowania;
    if (aktualny_typ_serializacji == binarna) {
        QDataStream strumien(dane);
        strumien >> wynik_rozkodowania.ustawienia_pid >>
            wynik_rozkodowania.ustawienia_arx >>
            wynik_rozkodowania.ustawienia_generatora;
    } else {
        QString zwykly_tekst = QString::fromUtf8(dane);
        QStringList wartosci = zwykly_tekst.split('|');
        if (wartosci.size() >= 3) {
            wynik_rozkodowania.ustawienia_pid = wartosci[0];
            wynik_rozkodowania.ustawienia_arx = wartosci[1];
            wynik_rozkodowania.ustawienia_generatora = wartosci[2];
        }
    }
    return wynik_rozkodowania;
}
```

Tryb binarny

```

void communication_protocol::przetworz_strumien(QByteArray &surowe_dane) {
    while (!surowe_dane.isEmpty()) {
        if (aktualny_typ_serializacji == binarna) {
            QDataStream strumien(surowe_dane);
            quint32 odczytany_typ, odczytany_rozmiar, wyjsciowy_numer_probki;

            strumien >> odczytany_typ >> odczytany_rozmiar >> wyjsciowy_numer_probki;

            if (strumien.status() != QDataStream::Ok)
                return;

            int waga_naglowka_w_pamieci = 3 * sizeof(quint32);
            int calkowityKosz = waga_naglowka_w_pamieci + (int)odczytany_rozmiar;

            //! czekamy aż przyjdzie cała zawartość pakietu - nie usuwać
            if (surowe_dane.size() < calkowityKosz)
                return;

            QByteArray wyjsciowe_dane =
                | surowe_dane.mid(waga_naglowka_w_pamieci, odczytany_rozmiar);
            surowe_dane.remove(0, calkowityKosz);

            typ_wiadomosci wyjsciowy_typ = static_cast<typ_wiadomosci>(odczytany_typ);
            if (wyjsciowy_typ == ramka_symulacji) {
                emit odebrano_symulacje(wyjsciowy_numer_probki,
                    | parsuj_dane_symulacji(wyjsciowe_dane));
            } else if (wyjsciowy_typ == komenda_sterujaca) {
                emit odebrano_sterowanie(wyjsciowy_numer_probki,
                    | parsuj_dane_sterujace(wyjsciowe_dane));
            } else if (wyjsciowy_typ == konfiguracja_uar) {
                emit odebrano_konfiguracje(wyjsciowy_numer_probki,
                    | parsuj_dane_konfiguracyjne(wyjsciowe_dane));
            }
        }
    }
}

```

Tryb tekstowy

```

    } else {
        QTextStream strumien(surowe_dane);
        uint odczytany_typ, odczytany_rozmiar, wyjsciowy_numer_probki;

        strumien >> odczytany_typ >> odczytany_rozmiar >> wyjsciowy_numer_probki;
        if (strumien.status() != QTextStream::Ok)
            return;

        int koniec_linii_index = surowe_dane.indexOf('\n');
        if (koniec_linii_index == -1)
            return;

        int calkowityKosz = koniec_linii_index + 1 + odczytany_rozmiar;
        if (surowe_dane.size() < calkowityKosz)
            return;

        QByteArray wyjsciowe_dane =
            | surowe_dane.mid(koniec_linii_index + 1, odczytany_rozmiar);
        surowe_dane.remove(0, calkowityKosz);

        typ_wiadomosci wyjsciowy_typ = static_cast<typ_wiadomosci>(odczytany_typ);
        if (wyjsciowy_typ == ramka_symulacji) {
            emit odebrano_symulacje(wyjsciowy_numer_probki,
                | parsuj_dane_symulacji(wyjsciowe_dane));
        } else if (wyjsciowy_typ == komenda_sterujaca) {
            emit odebrano_sterowanie(wyjsciowy_numer_probki,
                | parsuj_dane_sterujace(wyjsciowe_dane));
        } else if (wyjsciowy_typ == konfiguracja_uar) {
            emit odebrano_konfiguracje(wyjsciowy_numer_probki,
                | parsuj_dane_konfiguracyjne(wyjsciowe_dane));
        }
    }
}
}
}

```

NETWORK_HUB

Sekcja Public

```
class network_hub : public QObject {
    Q_OBJECT
public:
    explicit network_hub(QObject* parent = nullptr);
    ~network_hub(){rozlacz();}

    void uruchom_jako_serwer(quint16 port);
    void uruchom_jako_klient(const QString& ip, quint16 port);
    void rozlacz();

    QString get_serwer_ip();
    QString get_serwer_gniazdo();

    // wysyłanie 3 bloków symulacji
    void wyslij_zrzut_konfiguracji(uint32_t numer_probki, RegulatorPID* pid, ModelARX* arx, GenSyg* gen);

    // wysyłanie samych wyjść/wejść
    void wyslij_ramke(uint32_t numer_probki, double wejscie_obiektu_u, double wyjscie_obiektu_y, double wartosc_zadana_w);

    // dla startu/stopu/resetu i zmiany interwału - do sprawdzenia
    void wyslij_komende(uint32_t numer_probki, const QString& komenda, int interwal);

    void transmituj_konfiguracje(uint32_t numer_probki, const communication_protocol::dane_konfiguracyjne& pakiet);
    void transmituj_ramke(uint32_t numer_probki, const communication_protocol::dane_symulacji& pakiet);
```

Sekcja Signals & Slots

```
signals:
    void odebrano_symulacje(uint32_t numer_probki, communication_protocol::dane_symulacji dane);
    void odebrano_sterowanie(uint32_t numer_probki, communication_protocol::dane_sterujace dane);
    void odebrano_konfiguracje(uint32_t numer_probki, communication_protocol::dane_konfiguracyjne dane);
    void polaczono();
    void rozlaczono();

private slots:
    void na_nowe_polaczenie();
    void na_nowe_dane();
    void na_rozlaczenie();
```

Sekcja Private

```
private:
    QString formatuj_pid(RegulatorPID* pid);
    QString formatuj_arx(ModelARX* arx);
    QString formatuj_wektor(const std::vector<double>& wektor);
    QString formatuj_gen_syg(GenSyg* gen);

    void transmituj_komende(uint32_t nr_probki, const communication_protocol::dane_sterujace& pakiet);

    QTcpServer* m_serwer;
    QTcpSocket* m_gniazdo;
    communication_protocol m_protokol;
    QByteArray m_bufor;
};
```

Sekcja rozruchowa

```
network_hub::network_hub(QObject* parent)
    : QObject(parent), m_serwer(nullptr), m_gniazdo(nullptr)
{
    connect(&m_protokol, &communication_protocol::odebrano_symulacje, this, &network_hub::odebrano_symulacje);
    connect(&m_protokol, &communication_protocol::odebrano_sterowanie, this, &network_hub::odebrano_sterowanie);
    connect(&m_protokol, &communication_protocol::odebrano_konfiguracje, this, &network_hub::odebrano_konfiguracje);
}

void network_hub::uruchom_jako_serwer(quint16 port) {
    rozlacz();
    m_serwer = new QTcpServer(this);
    connect(m_serwer, &QTcpServer::newConnection, this, &network_hub::na_nowe_polaczenie);
    if (m_serwer->listen(QHostAddress::Any, port)) //any by było widoczne
    }
}

void network_hub::uruchom_jako_klient(const QString& ip, quint16 port) {
    rozlacz();
    m_gniazdo = new QTcpSocket(this);
    connect(m_gniazdo, &QTcpSocket::connected, this, &network_hub::polaczono);
    connect(m_gniazdo, &QTcpSocket::disconnected, this, &network_hub::na_rozlaczenie);
    connect(m_gniazdo, &QTcpSocket::readyRead, this, &network_hub::na_nowe_dane);
    m_gniazdo->connectToHost(ip, port);
}
```

```

void network_hub::rozlacz() {
    if (m_gniazdo) {
        m_gniazdo->disconnectFromHost();
        m_gniazdo->deleteLater();
        m_gniazdo = nullptr; //o tym zapomniałem
    }
    if (m_serwer) {
        m_serwer->close();
        m_serwer->deleteLater();
        m_serwer = nullptr; // o tym też
    }
}

void network_hub::na_nowe_polaczenie() {
    if (m_serwer && m_serwer->hasPendingConnections()) {
        if (m_gniazdo) {
            m_gniazdo->deleteLater();
        }
        m_gniazdo = m_serwer->nextPendingConnection();
        connect(m_gniazdo, &QTcpSocket::disconnected, this, &network_hub::na_rozlaczenie);
        connect(m_gniazdo, &QTcpSocket::readyRead, this, &network_hub::na_nowe_dane);
        emit polaczono();
    }
}

void network_hub::na_nowe_dane() {
    if (!m_gniazdo) return;
    m_bufor.append(m_gniazdo->readAll());
    m_protokol.przetworz_strumien(m_bufor);
}

void network_hub::na_rozlaczenie() {
    m_bufor.clear();
    emit rozlaczono();
}

```

Formatowanie

```
QString network_hub::formatuj_wektor(const std::vector<double>& wektor) {
    QStringList tekstowe;
    for (double w : wektor) {
        tekstowe.append(QString::number(w, 'f', 2));
    }
    QString wynik = "[";
    wynik.append(tekstowe.join(", "));
    wynik.append("]");

    return wynik;
}

QString network_hub::formatuj_pid(RegulatorPID* pid) {
    if (!pid) return "Brak";
    return QString("k=%1;Ti=%2;Td=%3").arg(pid->get_k()).arg(pid->get_Ti()).arg(pid->get_Td());
}

QString network_hub::formatuj_arx(ModelARX* arx) {
    if (!arx) return "Brak";
    return QString("A=%1;B=%2;opz=%3")
        .arg(formatuj_wektor(arx->getWspolczynnikiA()))
        .arg(formatuj_wektor(arx->getWspolczynnikiB()))
        .arg(arx->getOpoznienieTransportowe());
}

QString network_hub::formatuj_gen_syg(GenSyg* gen) {
    if (!gen) return "Brak";
    QString typNazwa = (gen->get_typ() == GenSyg::TypSyg::Sinusoida) ? "Sinusoida" : "Prostokat";
    return QString("Typ=%1;Trz=%2;Tt=%3;A=%4;S=%5")
        .arg(typNazwa)
        .arg(gen->get_Trz())
        .arg(gen->get_Tt())
        .arg(gen->get_A())
        .arg(gen->get_S());
}
```

Pakowanie

```
void network_hub::wyslij_zrzut_konfiguracji(uint32_t numer_probki, RegulatorPID* pid, ModelARX* arx, GenSyg* gen) {
    communication_protocol::dane_konfiguracyjne kompilacja;

    kompilacja.ustawienia_pid = formatuj_pid(pid);
    kompilacja.ustawienia_arx = formatuj_arx(arx);
    kompilacja.ustawienia_generatora = formatuj_gen_syg(gen);

    transmituj_konfiguracje(numer_probki, kompilacja);
}

void network_hub::wyslij_ramke(uint32_t numer_probki, double wejscie_obiektu_u, double wyjscie_obiektu_y, double wartosc_zadana_w) {
    communication_protocol::dane_symulacji ramka;
    ramka.sygnal_sterujacy = wejscie_obiektu_u;
    ramka.wartosc_regulowana = wyjscie_obiektu_y;
    ramka.wartosc_zadana = wartosc_zadana_w;

    transmituj_ramke(numer_probki, ramka);
}

void network_hub::wyslij_komende(uint32_t numer_probki, const QString& komenda, int interwal) {
    communication_protocol::dane_sterujace kom;
    kom.polecenie = komenda;
    kom.interwal = interwal;

    transmituj_komende(numer_probki, kom);
}
```

Transmitowanie

```
void network_hub::transmituj_konfiguracje(uint32_t nr, const communication_protocol::dane_konfiguracyjne& pakiet) {
    if (m_gniazdo && m_gniazdo->isOpen()) {
        QByteArray dane = m_protokol.stworz_wiadomosc_konfiguracyjna(nr, pakiet);
        m_gniazdo->write(dane);
        m_protokol.zarejestruj_wyslanie_pakietu();
    }
}

void network_hub::transmituj_ramke(uint32_t nr, const communication_protocol::dane_symulacji& pakiet) {
    if (m_gniazdo && m_gniazdo->isOpen()) {
        QByteArray dane = m_protokol.stworz_wiadomosc_symulacji(nr, pakiet);
        m_gniazdo->write(dane);
        m_protokol.zarejestruj_wyslanie_pakietu();
    }
}

void network_hub::transmituj_komende(uint32_t nr, const communication_protocol::dane_sterujace& pakiet) {
    if (m_gniazdo && m_gniazdo->isOpen()) {
        QByteArray dane = m_protokol.stworz_wiadomosc_sterujaca(nr, pakiet);
        m_gniazdo->write(dane);
        m_protokol.zarejestruj_wyslanie_pakietu();
    }
}
```

Klasa komunikacji

```
#include "Centrala.h"
#include "communication_protocol.h"
#include "Hub/network_hub.h"

class Komunikacja
{
private:
    Centrala m_centrala{};
    network_hub hub;

public:

    Komunikacja();
    //getter
    Centrala& getCentrala(){return m_centrala;}
    network_hub& get_hub(){return hub;}
    //metody->serializacja
    communication_protocol::dane_konfiguracyjne PID_ARX_do_wyslania_tekstowe();
    void odbior_tekstowe(const communication_protocol::dane_konfiguracyjne& dane);
    communication_protocol::dane_konfiguracyjne PID_ARX_do_wyslania_binarnka();
    void odbior_binarnka(const communication_protocol::dane_konfiguracyjne& dane);
};
```

Serializacja

Binarna

```
communication_protocol::dane_konfiguracyjne Komunikacja::PID_ARX_do_wyslania_binarnka()
{
    communication_protocol::dane_konfiguracyjne dane;

    //PID
    QByteArray ramka_PID;
    QDataStream stream_PID(&ramka_PID, QIODevice::WriteOnly);

    stream_PID<<m_centrala.getPID_K();
    stream_PID<<m_centrala.getPID_Ti();
    stream_PID<<m_centrala.getPID_Td();

    dane.ustawienia_pid=QString::fromLatin1(ramka_PID.toBase64());
    //Lation1->zawiera ASCII oraz literki z np francuskiego

    //ARX
    QByteArray ramka_ARX;
    QDataStream stream_ARX(&ramka_ARX, QIODevice::WriteOnly);

    stream_ARX << m_centrala.getWartoscZaklocenia();
    stream_ARX << m_centrala.getOpoznienieTransportowe();
    stream_ARX << m_centrala.isRegulacjaWlaczona();
    stream_ARX << m_centrala.isSterowanieWlaczona();
    stream_ARX << m_centrala.getMaxRegulowania();
    stream_ARX << m_centrala.getMinRegulowania();
    stream_ARX << m_centrala.getMaxSterowania();
    stream_ARX << m_centrala.getMinSterowania();

    std::vector<double> A = m_centrala.getWspolczynnikiA();
    stream_ARX << (quint32)A.size();
    for(double val: A) {
        stream_ARX << val;
    }
    std::vector<double> B = m_centrala.getWspolczynnikiB();
    stream_ARX << (quint32)B.size();
    for(double val: B) {
        stream_ARX << val;
    }

    dane.ustawienia_arx=QString::fromLatin1((ramka_ARX.toBase64()));
    return dane;
}
```

```
void Komunikacja::odbior_binarka(const communication_protocol::dane_konfiguracyjne &dane)
{
    //rozpakowanie PID
    QByteArray ramka_Pid = QByteArray::fromBase64(dane.ustawienia_pid.toLatin1());
    QDataStream stream_Pid(ramka_Pid);

    double p, i, d;
    stream_Pid >> p;
    stream_Pid >> i;
    stream_Pid >> d;

    m_centrala.setPID_K(p);
    m_centrala.setPID_Ti(i);
    m_centrala.setPID_Td(d);
}
```

```

//ARX
QByteArray ramka_Arx = QByteArray::fromBase64(dane.ustawienia_arx.toLatin1());
QDataStream streamArx(ramka_Arx);

bool is_reg_wlacz, is_ster_wlacz;
double min_reg, max_reg, min_ster, max_ster, szumy;
int opoznienie;
std::vector<double> A, B;
quint32 wsp_a, wsp_b;

streamArx >> szumy;
streamArx >> opoznienie;
streamArx >> is_reg_wlacz;
streamArx >> is_ster_wlacz;
streamArx >> max_reg;
streamArx >> min_reg;
streamArx >> max_ster;
streamArx >> min_ster;

streamArx >> wsp_a;
for(quint32 i = 0; i < wsp_a; i++) {
    double a;
    streamArx >> a;
    A.push_back(a);
}

streamArx >> wsp_b;
for(quint32 i = 0; i < wsp_b; i++) {
    double b;
    streamArx >> b;
    B.push_back(b);
}

m_centrala.zastosujParametryARX(A, B, opoznienie, szumy, is_ster_wlacz, min_ster, max_ster, is_reg_wlacz, min_reg, max_reg);

```

Tekstowa

```
//Tekstowo
communication_protocol::dane_konfiguracyjne Komunikacja::PID_ARX_do_wyslania_tekstowe()
{
    communication_protocol::dane_konfiguracyjne dane;
    //PID
    QJsonObject pid;
    pid["p"] = m_centrala.getPID_K();
    pid["i"] = m_centrala.getPID_Ti();
    pid["d"] = m_centrala.getPID_Td();
    //DO dokumnetu
    dane.ustawienia_pid = QString(QJsonDocument(pid).toJson(QJsonDocument::Compact));

    //ARX
    QJsonObject arx;
    QJsonArray wsp_a;
    QJsonArray wsp_b;

    for (double val : m_centrala.getWspolczynnikiA()) {
        wsp_a.append(val);
    }
    for (double val : m_centrala.getWspolczynnikiB()) {
        wsp_b.append(val);
    }

    arx["wspolczynnikiA"] = wsp_a;
    arx["wspolczynnikiB"] = wsp_b;
    arx["opoznienie"] = m_centrala.getOpoznienieTransportowe();
    arx["szumy"] = m_centrala.getWartoscZaklocenia();
    arx["minSter"] = m_centrala.getMinSterowania();
    arx["maxSter"] = m_centrala.getMaxSterowania();
    arx["minReg"] = m_centrala.getMinRegulowania();
    arx["maxReg"] = m_centrala.getMaxRegulowania();
    arx["isSterowanieWlaczona"] = m_centrala.isSterowanieWlaczona();
    arx["isRegulacjaWlaczona"] = m_centrala.isRegulacjaWlaczona();

    dane.ustawienia_arx = QString(QJsonDocument(arx).toJson(QJsonDocument::Compact));

    return dane;
}
```

```

void Komunikacja::odbior_tekstowe(const communication_protocol::dane_konfiguracyjne& dane)
{
    //rozpakowanie Arx z dokumentu
    QJsonDocument docArx = QJsonDocument::fromJson(dane.ustawienia_arx.toUtf8());
    QJsonObject arx = docArx.object();

    QJsonArray wsp_a_odbior = arx["wspolczynnikiA"].toArray();
    std::vector<double> A;
    for (QJsonValue val : wsp_a_odbior) {?      ▲c++11 range-loop might detach Qt container (QJsonArray) [clazy-range-loop-detach]      ▲Missing reference in ran
    |     A.push_back(val.toDouble());
    }

    QJsonArray wsp_b_odbior = arx["wspolczynnikiB"].toArray();
    std::vector<double> B;
    for (QJsonValue val : wsp_b_odbior) {?      ▲c++11 range-loop might detach Qt container (QJsonArray) [clazy-range-loop-detach]      ▲Missing reference in ran
    |     B.push_back(val.toDouble());
    }

    int opoznienie = arx["opoznienie"].toInt();
    double szumy = arx["szumy"].toDouble();
    double min_sterowanie = arx["minSter"].toDouble();
    double max_sterowanie = arx["maxSter"].toDouble();
    double min_regulowana = arx["minReg"].toDouble();
    double max_regulowana = arx["maxReg"].toDouble();
    bool is_ster_wlacz = arx["isSterowanieWlaczona"].toBool();
    bool is_reg_wlacz = arx["isRegulacjaWlaczona"].toBool();

    m_centrala.zastosujParametryARX(A, B, opoznienie, szumy, is_ster_wlacz, min_sterowanie, max_sterowanie, is_reg_wlacz, min_regulowana, max_regulowana);
    // PID
    QJsonDocument docPid = QJsonDocument::fromJson(dane.ustawienia_pid.toUtf8());
    QJsonObject pid = docPid.object();
    m_centrala.setPID_K(pid["p"].toDouble());
    m_centrala.setPID_Ti(pid["i"].toDouble());
    m_centrala.setPID_Td(pid["d"].toDouble());
}

```

Spotkanie 2

Poprawa uchybień znalezionych na etapie
weryfikacji oraz walidacji

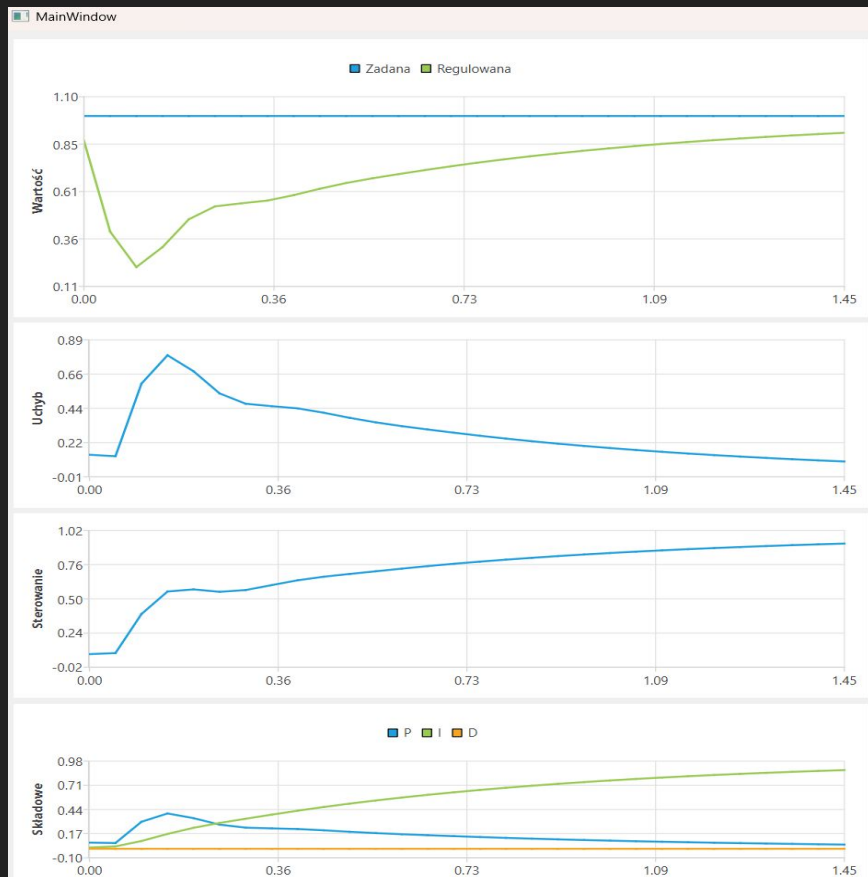
Brak zerowania buforów sygnałowych w modelu ARX w momencie resetu symulacji

```
void ModelARX::ResetPamieci()
{
    //wypelnienie 0.0 bufforów dla ARX
    fill(u_buffor.begin(),u_buffor.end(),0.0);
    fill(y_buffor.begin(),y_buffor.end(),0.0);
    fill(delay_buffor.begin(),delay_buffor.end(),0.0);
}

|
void ProstyUAR::ResetStanow()
{
    m_arx.ResetPamieci();
    m_regulatorPID.resetPamieci();
    w = 0.0; //wartosc zadana
    y = 0.0; //wartosc zmierzona z arx
    e = 0.0; //uchyb,  $e_i = w_i - y_{(i-1)}$ 
    u = 0.0; //wartosc sterowania,  $u_i = regulator(e_i)$ 
}
```

```
void Centrala::resetSymulacji()  
{  
  
    m_ProstyUAR.ResetStanow();  
    m_generatorSygnału.ResetPamieci();  
  
    m_ostatnie_w = 0.0;  
    m_ostatnie_y = 0.0;  
}
```

Przed zmianą



Po zmianie



Opóźnienie transportowe przyjmowane jest dopiero za drugim razem (należy 2 kliknąć Zastosuj, aby symulacja zareagowała)

```
void ProstyUAR::zastosujParametryARX(vector<double> A, vector<double> B, double opoznienie, double szum, bool włączSter, double minS, double maxS, double minR, double maxR)
{
    m_arx.set_opoznienieTransportowe(opoznienie);
    m_arx.setWspółczynnikA(A);
    m_arx.setWspółczynnikB(B);
    m_arx.setWartoscZakłocenia(szum);
    m_arx.włączSterowanie(włączSter);
    m_arx.setWartoscSterowania(minS, maxS);
    m_arx.włączRegulacje(włączReg);
    m_arx.setWartoscRegulowania(minR, maxR);
}
```

Łączenie

```
void network_hub::na_nowe_polaczenie() {  
    if (m_serwer && m_serwer->hasPendingConnections()) {  
        if (m_gniazdo) {  
            m_gniazdo->deleteLater();  
        }  
        m_gniazdo = m_serwer->nextPendingConnection();  
        connect(m_gniazdo, &QTcpSocket::disconnected, this, &network_hub::na_rozlaczenie);  
        connect(m_gniazdo, &QTcpSocket::readyRead, this, &network_hub::na_nowe_dane);  
        emit polaczono();  
    }  
}
```

```
connect(&komunikacja.get_network_hub(), &network_hub::polaczono, this, &MainWindow::Czy_polaczylo);
```

```
void MainWindow::Czy_polaczylo()  
{  
    ui->ip_adres_wyswietl_TextEdit->appendPlainText(komunikacja.get_network_hub().get_serwer_ip());  
    QMessageBox::information(this, "Info", "Połączono z serwerem");  
}
```

```
void MainWindow::on_server_radioButton_clicked()
{
    bool reg=false, arx=true;
    komunikacja.get_network_hub().uruchom_jako_serwer(1235);

    QString ip;
    ui->ip_adres_wyswietl_TextEdit->setReadOnly(false);
    ui->ip_adres_wyswietl_TextEdit->setPlainText(ip);
    //ustawienie kontrolek
    wylacz_wlacz_kontrolki(reg, arx);

    ARX_wysyl();
    PID_odbior();
    Gen_odbior();
    Interwal_odbior();
}
```

```
void MainWindow::on_klient_radioButton_clicked()
{
    bool ok, reg=true, arx=false;
    QString text = QInputDialog::getText(this, "Tytuł okna", "Podaj adres IP serwera:", QLineEdit::Normal, "127.0.0.1", &ok);

    if (ok && !text.isEmpty()) {
        komunikacja.get_network_hub().uruchom_jako_klient(text,1235);
    }
    //ustawienie kontrolek
    wylacz_wlacz_kontrolki(reg, arx);

    ARX_odbior();
    PID_wysyl();
    Gen_wysyl();
    Interwal_wysyl();
}
```

```
void MainWindow::wylacz_wlacz_kontrolki(bool reg, bool arx)
{
    //kontrola sym
    ui->okno->setEnabled(reg);
    ui->interwal->setEnabled(reg);
    //gen
    ui->generatorCombo->setEnabled(reg);
    ui->amplituda->setEnabled(reg);
    ui->okres->setEnabled(reg);
    ui->wypelnienie->setEnabled(reg);
    ui->skladowaStala->setEnabled(reg);
    //PID
    ui->wzmocnienie->setEnabled(reg);
    ui->stalaI->setEnabled(reg);
    ui->stalaD->setEnabled(reg);
    //reset I && D
    ui->resetRozniczki->setEnabled(reg);
    ui->resetCalki->setEnabled(reg);
    //calka
    ui->rodzajCalkowaniaCombo->setEnabled(reg);
    //ograniczenia
    ui->ograniczenia->setEnabled(reg);
    ui->ograniczeniaMax->setEnabled(reg);
    ui->ograniczeniaMin->setEnabled(reg);
    //arx
    ui->parametryARX->setEnabled(arx);
}
```

```
void MainWindow::wylacz_wlacz_kontrolki_sieciowe(bool b)
{
    ui->klient_radioButton->setEnabled(b);
    ui->server_radioButton->setEnabled(b);
    ui->wybor_serializacji_comboBox->setEnabled(b);
    ui->lcdNumber->setEnabled(b);
    ui->ip_adres_wyswietl_TextEdit->setEnabled(b);
    ui->ip_adres_wyswietl_TextEdit->clear();
}
```

```
void MainWindow::on_comboBox_currentTextChanged(const QString &arg1)
{
    bool b=true, arx=true, pid=true;

    if(arg1=="Sieciowy"){
        |   wylacz_wlacz_kontrolki_sieciowe(b);
    }
    else{
        b=false;

        wylacz_wlacz_kontrolki_sieciowe(b);
        ui->klient_radioButton->setChecked(b);
        |   ui->server_radioButton->setChecked(b);
        wylacz_wlacz_kontrolki(arx, pid);
    }
}
```

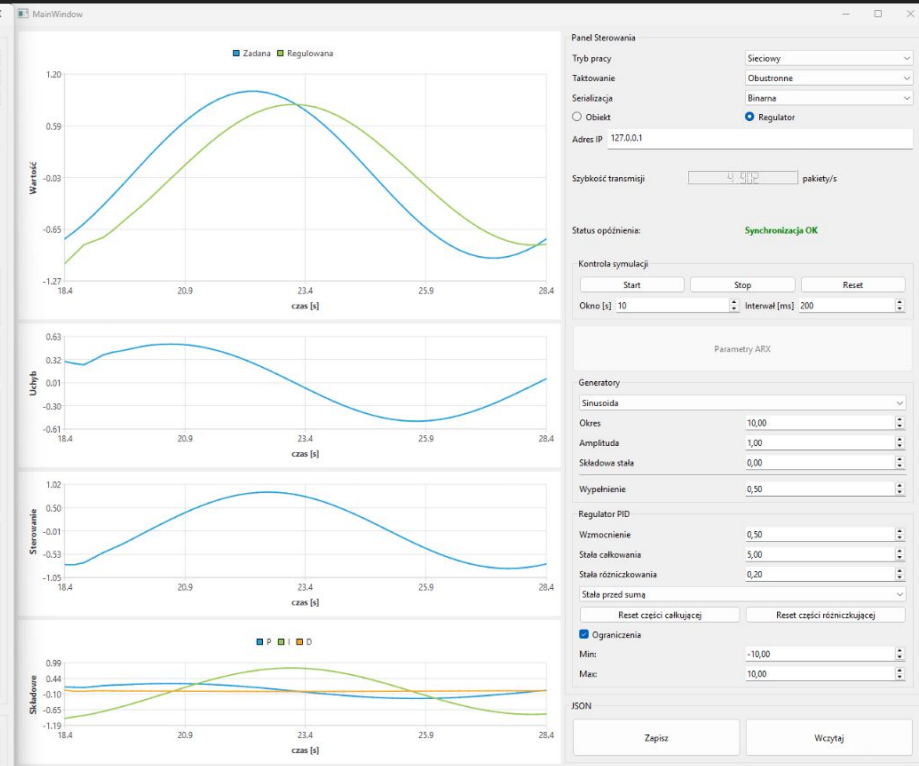
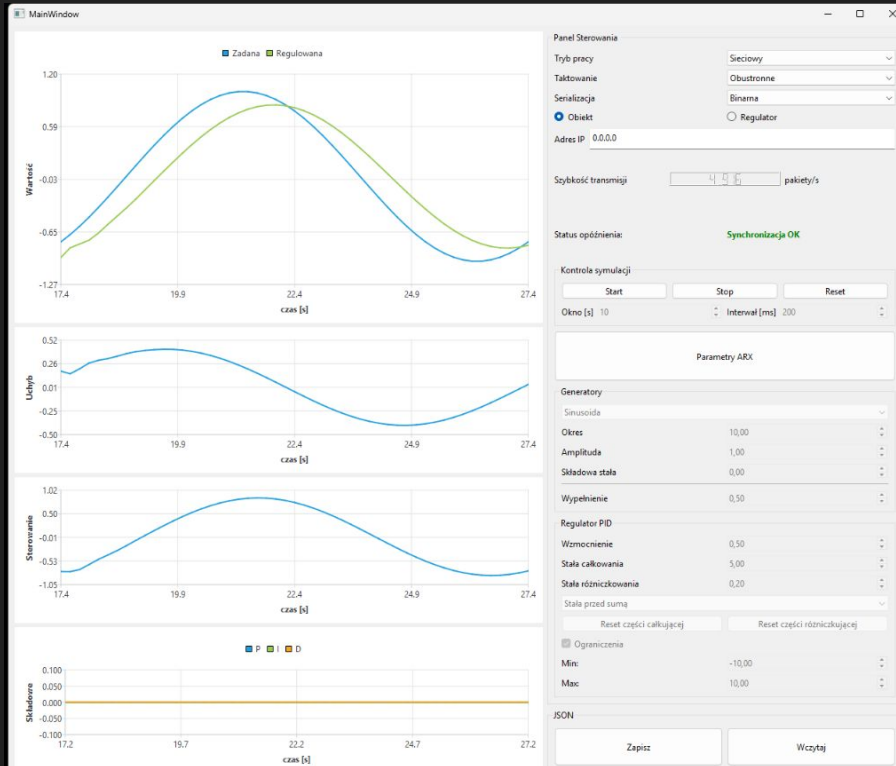
Spotkanie 3

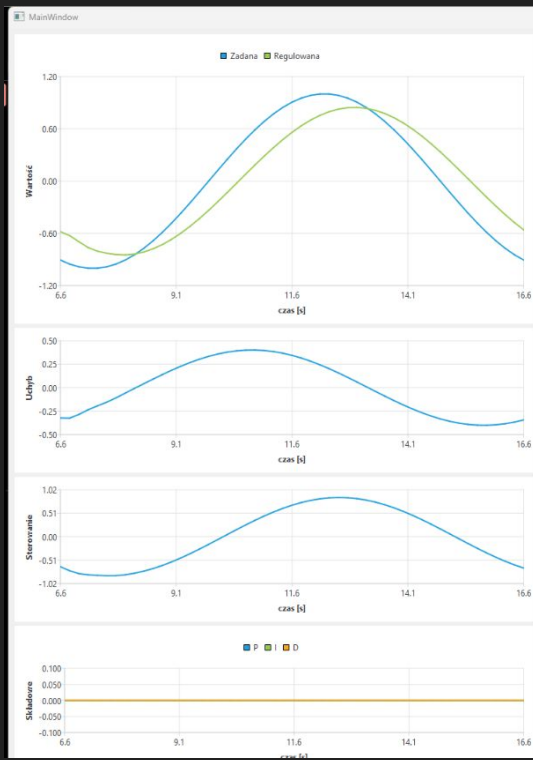
Kontrola pakietu oraz radzenie sobie z
opóźnieniami

```
bool communication_protocol::czy_nadaza(int interwal_symulacji_ms) const {  
    if (czas_ostatniego_pakietu_ms == 0)  
        return true; // dla 1 pakietu -- i am dump  
  
    qint64 obecnie_na zegarze_ms = QDateTime::currentMsecsSinceEpoch();  
    qint64 pustka_od_ostatniego_pakietu =  
        obecnie_na zegarze_ms - czas_ostatniego_pakietu_ms;  
  
    // tolerujemy laga w przeciagu półtora kroku interwału - profilaktycznie  
    return pustka_od_ostatniego_pakietu <= (interwal_symulacji_ms * 1.5);  
}
```

```
bool communication_protocol::czy_zsynchronizowane(
    uint32_t lokalny_numer_probki, uint32_t zdalny_numer_probki,
    uint32_t maksymalne_odchylenie) const {
    uint32_t rozstrzelenie = (lokalny_numer_probki > zdalny_numer_probki)
        ? (lokalny_numer_probki - zdalny_numer_probki)
        : (zdalny_numer_probki - lokalny_numer_probki);

    return rozstrzelenie <= maksymalne_odchylenie;
}
```





Panel Sterowania

Tryb pracy: Sieciowy

Taktowanie: Jednostronne

Serializacja: Binarna

☒ Obiekt ☐ Regulator

Adres IP: 0.0.0.0

Szybkość transmisji: 4 000 pakiety/s

Status opóźnienia: **Opóźnienie / Utrata**

Kontrola symulacji: Start Stop Reset

Okno [s]: 10 Interval [ms]: 200

Parametry ARX

Generatory: Sinusoide

Okres: 10,00

Amplituda: 1,00

Składowa stała: 0,00

Wypełnienie: 0,50

Regulator PID: Wzmocnienie: 0,50

Stała całkowania: 5,00

Stała różniczkowania: 0,20

Stała przed sumą

Reset części całkującej Reset części różniczkującej

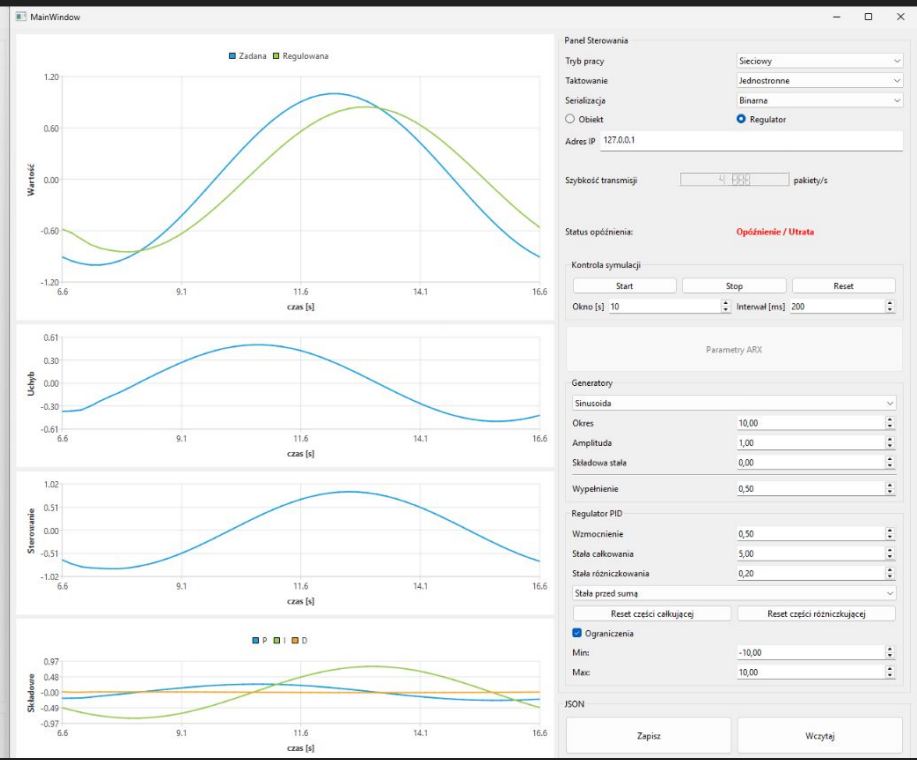
☐ Ograniczenia

Min: -10,00

Max: 10,00

JSON

Zapisz Wczytaj



Panel Sterowania

Tryb pracy: Sieciowy

Taktowanie: Jednostronne

Serializacja: Binarna

☐ Obiekt ☒ Regulator

Adres IP: 127.0.0.1

Szybkość transmisji: 4 000 pakiety/s

Status opóźnienia: **Opóźnienie / Utrata**

Kontrola symulacji: Start Stop Reset

Okno [s]: 10 Interval [ms]: 200

Parametry ARX

Generatory: Sinusoide

Okres: 10,00

Amplituda: 1,00

Składowa stała: 0,00

Wypełnienie: 0,50

Regulator PID: Wzmocnienie: 0,50

Stała całkowania: 5,00

Stała różniczkowania: 0,20

Stała przed sumą

Reset części całkującej Reset części różniczkującej

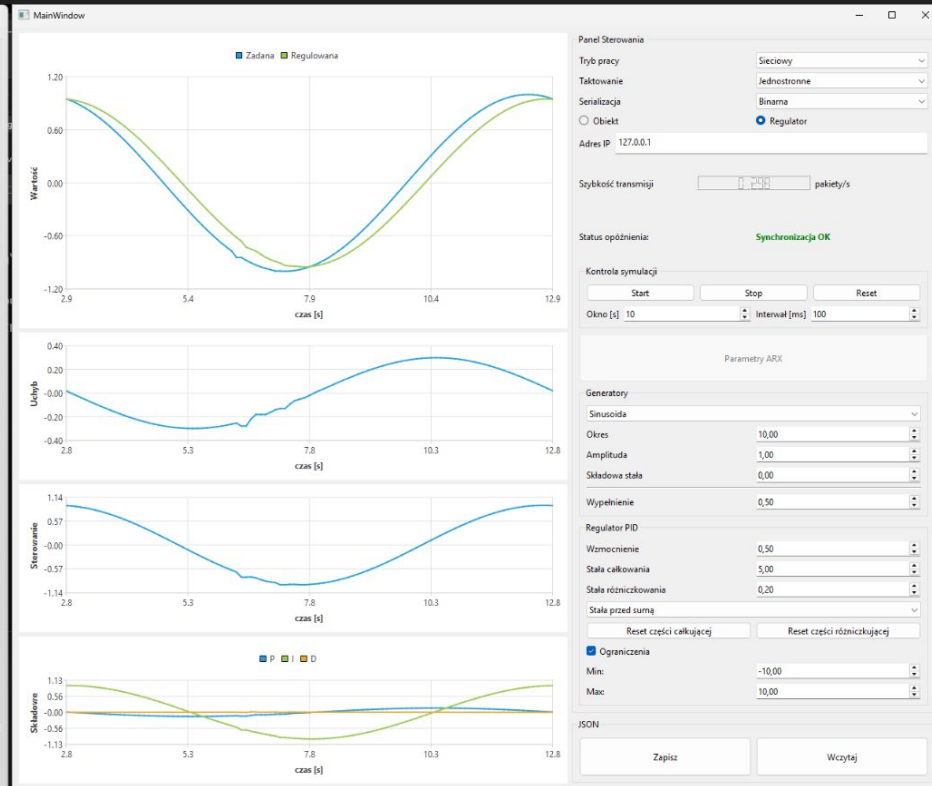
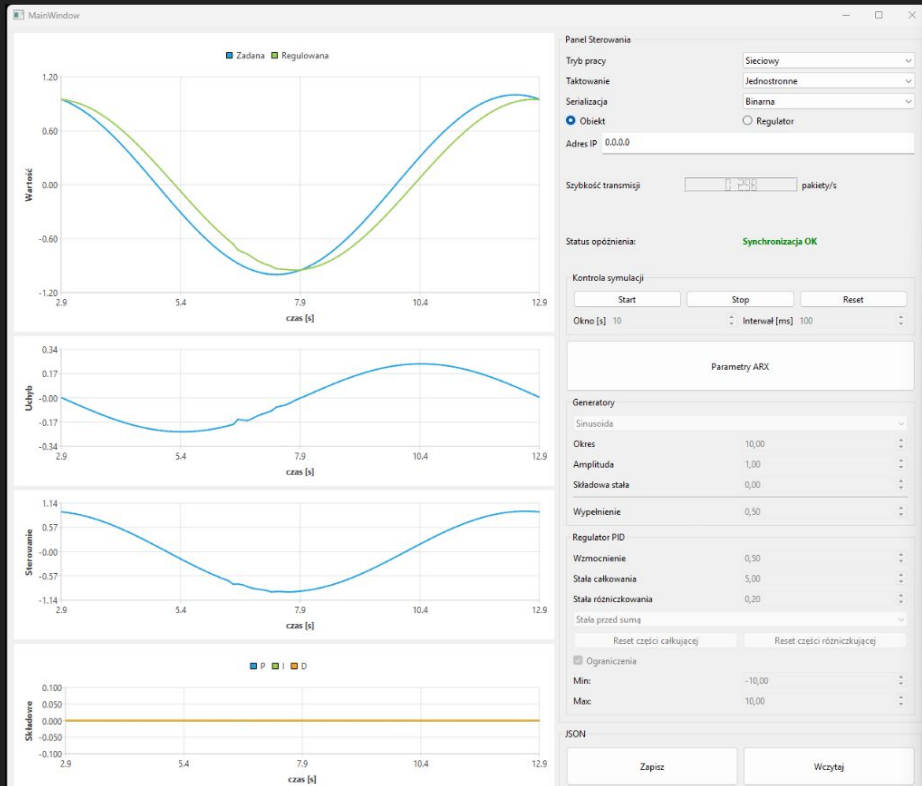
☒ Ograniczenia

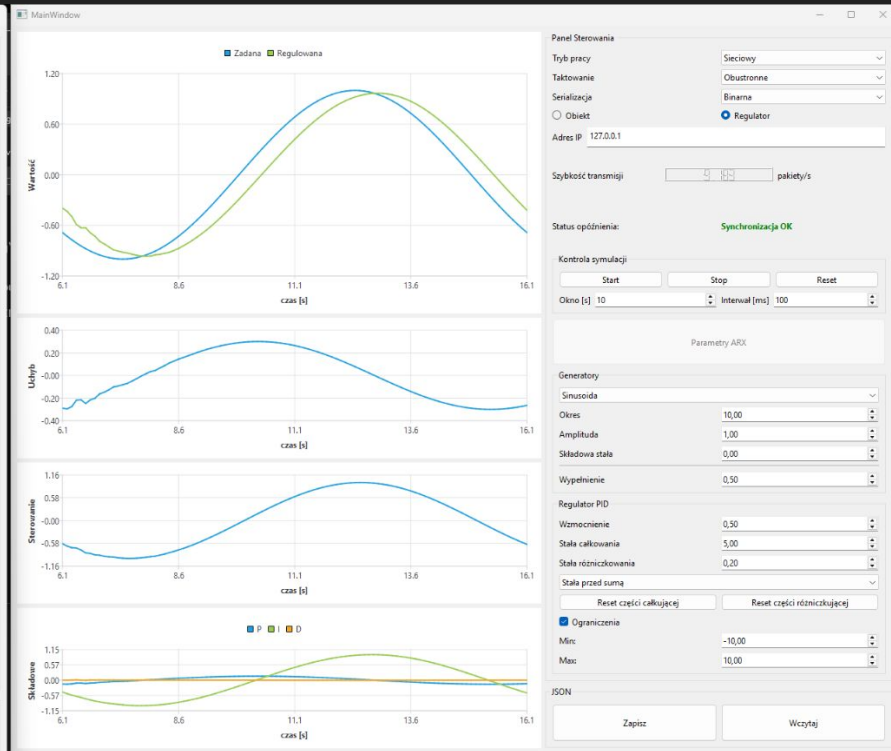
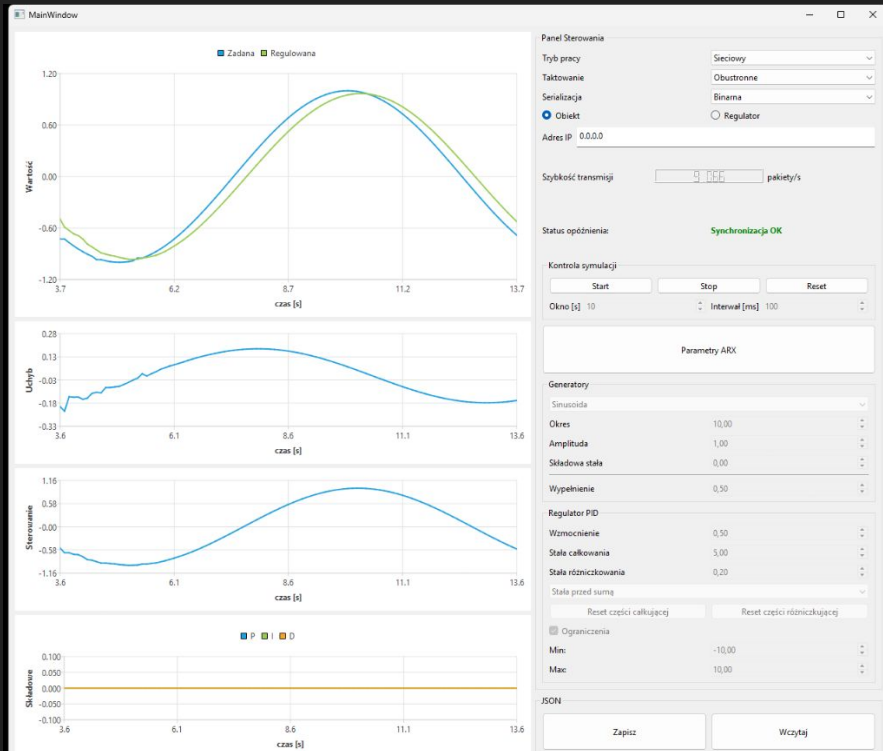
Min: -10,00

Max: 10,00

JSON

Zapisz Wczytaj





Omówienie Protokołu Komunikacyjnego

Główne cechy:

- Oparty na strumieniach TCP
- Obsługuje trzy kluczowe kategorie informacji:
 1. Ciągłe dane z symulacji układu (przesyłane z dużą częstotliwością).
 2. Asynchroniczne komendy sterujące (zdarzenia i zmiany parametrów).
 3. Jednorazowe pakiety konfiguracji obiektów (np.parametry początkowe).

Zarys Konceptyjny - Typy Wiadomości

Protokół definiuje enum `typ_wiadomosci`, który kategoryzuje przesyłane pakiety na trzy główne typy:

1. `ramka_symulacji`
2. `komenda_sterujaca`
3. `konfiguracja_uar`

1. ramka_symulacji

- Przeznaczenie: Ciągłe przesyłanie sygnałów z układu.
- Zawartość: Wartość zadana, uchyb / wartość regulowana, sygnał sterujący.
- Charakterystyka: Najczęściej przesyłana ramka, kluczowa dla wykresów i monitoringu układu.

2. komenda_sterujaca

- Przeznaczenie: Asynchroniczne zdarzenia kontrolne.
- Zawartość: Komendy - START, STOP, RESET.
- zmiana parametrów interwał próbkowania / rozmiar okna.

3. konfiguracja_uar

- Przeznaczenie: Inicjalizacja i wymiana parametrów.
- Zawartość: Nastawy regulatora PID, parametry modelu ARX, ustawienia generatora sygnałów.

Zarządzanie Przepływem i Synchronizacja

1. Pomiary Wydajności:

- Wykorzystanie obiektu QElapsedTimer do precyzyjnego pomiaru wskaźnika PPS.

Pozwala to na bieżąco monitorować przepustowość łącza.

2. Weryfikacja Synchronizacji:

- Każda ramka symulacji posiada własny unikalny numer próbki.
- Mechanizm dopuszcza pewne odchylenie numerów próbek.

Architektura Pamięciowa - Niezmienny Nagłówek

Budowa Nagłówka (Stały rozmiar: 12 Bajtów / 96 bitów):

Składa się z trzech 32-bitowych słów ($3 * \text{sizeof}(\text{uint32})$):

1. Typ wiadomości (4 bajty) - identyfikuje czy ramka to symulacja/ komenda / konfiguracja

(enum: 1, 2, 3).

2. Rozmiar danych (4 bajty) - określa dokładną wielkość payloadu. Jest to kluczowe dla parsera by wiedział, ile dokładnie bajtów musi jeszcze odebrać.

3. Numer próbki (4 bajty) - wykorzystywany do logiki synchronizacji - weryfikacji ciągłości.

Tryb Binarny - Konstrukcja Pakietów

Budowa pakietów w trybie binarnym (Payload):

1. Ramka Symulacji (Całość: 36 Bajtów)

- Nagłówek (12 bajtów)

- Payload (24 bajty): 3 zmienne **double** (sygnal_sterujacy, wartosc_regulowana, wartosc_zadana).

Po 8 bajtów (64 bity).

Tryb Binarny - Konstrukcja Pakietów

2. Komenda Sterująca (Całość: 24 Bajty + długość znaków w stringu)

- Nagłówek (12 bajtów)
- Payload:
 - + QString : 4 bajty na element stringa
 - + “interwał” : qint32 (4 bajty)
 - + “okno” : qint32 (4 bajty)

3. Konfiguracja UAR

- Nagłówek (12 bajtów)
- Payload: Trzy obiekty typu QString , parametry: PID, modele ARX i nastawy generatora. jak w Komendzie sterującej.

Tryb Tekstowy - Konstrukcja Pakietów

Budowa pakietów w trybie tekstowym:

1. Ramka Symulacji

- Zawsze binarny nagłówek (12 bajtów).
- Payload to sformatowany jako ciąg znaków string, np. "0.245;12.5;20.0".

Separatorem jest ;

Tryb Tekstowy - Konstrukcja Pakietów

2. Komenda Sterująca

- Binarny nagłówek (12 bajtów).
- Payload w określonym formacie: "{polecenie};{interwal};{okno}" np. "START;100;5".

3. Konfiguracja UAR

- Binarny nagłówek (12 bajtów).
- Payload w formacie: "{ustawienia_pid}||{ustawienia_arx}||{ustawienia_generатора}".

Przykłady dla ramka_symulacji

// 1. ramka_symulacji BINARNA u=1.0, y=2.0, w=2.2, probka=100		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 01	Int32: 1
Rozmiar danych	00 00 00 18	Int32: 24 bajty
Numer próbki	00 00 00 64	Int32: 100
Sygnał sterujący	3F F0 00 00 00 00 00 00	Double: 1.0 (u)
Wartość regulowana	40 00 00 00 00 00 00 00	Double: 2.0 (y)
Wartość zadana	40 01 99 99 99 99 99 9A	Double: 2.2 (w)

// 1. ramka_symulacji TEKSTOWA u=1, y=2, w=2.2, probka=100		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 01	Int32: 1
Rozmiar danych	00 00 00 07	Int32: 7 bajtów
Numer próbki	00 00 00 64	Int32: 100
Payload	31 3B 32 3B 32 2E 32	ASCII: "1;2;2.2"

Przykłady dla komenda_sterujaca

// 2. komenda_sterujaca BINARNA START, interwal=100, okno=10, probka=0		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 02	Int32: 2
Rozmiar danych	00 00 00 12	Int32: 18 bajtów
Numer próbki	00 00 00 00	Int32: 0
Polecenie	00 00 00 0A + 00 53 00 54 00 41 00 52 00 54	"START"
Interwał	00 00 00 64	Int32: 100
Okno	00 00 00 0A	Int32: 10

// 2. komenda_sterujaca TEKSTOWA START, interwal=100, okno=10, probka=0		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 02	Int32: 2
Rozmiar danych	00 00 00 0C	Int32: 12 bajtów
Numer próbki	00 00 00 00	Int32: 0
Payload	53 54 41 52 54 3B 31 30 30 3B 31 30	ASCII: "START;100;10"

Przykłady dla konfiguracja_uar

// 3. konfiguracja_uar BINARNA pid="kp=1.2", arx="a1=0.8", gen="amp=1.0", probka=0		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 03	Int32: 3
Rozmiar danych	00 00 00 38	Int32: 56 bajtów
Numer próbki	00 00 00 00	Int32: 0
Ustawienia PID	00 00 00 0E 00 6B 00 70 00 3D 00 31 00 2E 00 32	kp=1.2
Ustawienia ARX	00 00 00 0C 00 61 00 31 00 3D 00 30 00 2E 00 38	a1=0.8
Ustawienia GEN	00 00 00 0E 00 61 00 6D 00 70 00 3D 00 31 00 2E 00 30	amp=1.0

// 3. konfiguracja_uar TEKSTOWA pid="kp=1.2", arx="a1=0.8", gen="amp=1.0", probka=0		
Pole	Wartość	Opis
Typ wiadomości	00 00 00 03	Int32: 3
Rozmiar danych	00 00 00 12	Int32: 18 bajtów
Numer próbki	00 00 00 00	Int32: 0
Payload	6B 70 3D 31 2E 32 7C 61 31 3D 30 2E 38 7C 61 6D 70 3D 31 2E 30	ASCII: kp=1.2 a1=0.8 amp=1.0